

A verified CAD algorithm and decision procedure in PVS

An overview of the `pvs_cad` library and its proof strategies `(cad)` and `(cad-qe)`

J. Tanner Slagel*

Code and proofs: https://github.com/j-tanner-slagel/pvs_cad

In short. `pvs_cad` is a PVS library, with the proof strategies `(cad)` and `(cad-qe)`, that decides the first-order theory of the real numbers and eliminates quantifiers: formulas built from polynomial equations and inequalities with rational coefficients, the connectives AND, OR, NOT, IMPLIES and IFF, and quantifiers over real variables. An executable PVS function computes a cylindrical algebraic decomposition (CAD) from Collins’s projection operator, and the library proves, for every family of polynomials in every number of variables and with no run-time certificate, that the cells of $\mathbb{R}^n$ cover it and are disjoint, that the cells of every level are connected and first-order definable and stack in cylinders over continuous, strictly ordered root functions with definable graphs, and that each cell of $\mathbb{R}^n$ carries one sign vector of the input polynomials (`col_cells`); `col_found` returns this CAD as data, at least one record with an exact sample point for every nonempty cell of $\mathbb{R}^n$ and none for any other, and those records decide every sentence over the polynomials in the same variables, in the same order. A set is definable by a first-order formula with rational coefficients exactly when a quantifier-free one defines it (`fod_qfd`, Tarski–Seidenberg over $\mathbb{Q}$), and every cell, at every level, is quantifier-free definable (`col_cells_sa`). In a theory that imports `pvs_cad`, `(cad)` decides a closed prenex formula over `real` whose atoms compare polynomials in its bound variables by `decide8`, the Collins run (after a witness search), and any other first-order formula over the reals, one with other real terms included, by `decide_g`. Both are proved *sound* (an answer is always the truth) and *complete* (they always answer). `(cad-qe)` replaces a formula with free variables by a quantifier-free one that holds at exactly the same points. Inside a proof, the strategies evaluate the procedure and turn its answer into a PVS proof that the kernel checks. The whole library, 210 theories and 3,175 formulas, and its 14 example theories, with 457 formulas more, are proved.

Using it. A theory needs one line, `IMPORTING cad@pvs_cad` (with the repository’s root on `PVS_LIBRARY_PATH`); for constants such as π and e , and for `sin`, `exp`, ... of constants, `IMPORTING cad@pvs_cad_num` instead, which adds NASALib’s interval arithmetic and the bounds of `trans_bounds`. Then, in a proof:

<code>(cad)</code>	decide the goal, a first-order sentence over the reals: a TRUE goal is proved; otherwise it stays, labelled <code>cad</code> , with a message saying it is FALSE
<code>(cad -1)</code> , <code>(cad 2)</code> , <code>(cad (-1 3))</code>	the same on a chosen hypothesis or goal (a FALSE hypothesis closes the goal), or on several formulas together
<code>(cad *)</code> , <code>(cad +)</code> , <code>(cad -)</code>	decide the sequent as a whole (the hypotheses imply the goals), or the goals or the hypotheses alone; other real terms, such as $f(c)$ or $\sqrt{c}$, become unknowns
<code>(cad-qe)</code> , <code>(cad-qe -1)</code>	replace a goal or hypothesis with free real terms by an equivalent quantifier-free formula
<code>(cad :cad-only? t)</code> , <code>(cad-direct)</code>	answer only through the verified CAD engine; decide without the witness search
<code>(cad-num)</code> , <code>(cad-facts)</code>	what <code>(cad)</code> falls back to for constants such as π , for <code>sqrt</code> , <code>abs</code> , <code>max</code> , <code>min</code> and quotients, and for <code>sin</code> , <code>exp</code> , ... of constants: decide with them pinned down; only add the exact facts

Formulas are taken as written: grouped binders, quantifiers inside NOT, AND, OR, IMPLIES and IFF,

*This report was written with Claude Code using Claude Opus 5.5. The `pvs_cad` library was developed the same way, with Claude Fable 5.1 and then Opus 5 and Opus 5.5; every proof in it is checked by PVS. Claude Code: <https://claude.com/claude-code>.

PVS's synonyms for the connectives, and binders over `posreal`, `nnreal`, `negreal`, `npreal` and `nzreal`. `cad/cad_examples/cad_showcase.pvs` has 43 commented examples, each with its proof and its time, and `cad/cad_examples/cad_forms_ex.pvs` 79 regression lemmas.

What it looks like

Real PVS sessions, recorded on 1 October 2026 in `cad_showcase`, which imports `pvs_cad`, with the prover's commentary and the simplifier's internal messages cut (the commands print the same here; their times are in the table under "What it decides in practice"). The quadratic formula as an existence theorem, written with a grouped binder (`((cad` proves it from its nested form; the timing table below gives the whole proof, `quadratic_formula)`:

```
quadratic_formula :
|-----
{1}  FORALL (a, b, c: real):
      EXISTS (x: real):
        a /= 0 AND b ^ 2 >= 4 * a * c IMPLIES a * x ^ 2 + b * x + c = 0
Rule? (cad)
Deciding 1 by cylindrical decision, witness first,
Q.E.D.
```

Continuity of x^2 at 0, the epsilon-delta way: three alternating quantifiers, binders over `posreal` and an implication inside, taken as written:

```
sq_continuous :
|-----
{1}  FORALL (e: posreal):
      EXISTS (d: posreal):
        FORALL (x: real): -d < x AND x < d IMPLIES x ^ 2 < e
Rule? (cad)
Deciding 1 by cylindrical decision, witness first,
Q.E.D.
```

`(cad *)` works on a whole sequent. After `(skeep)` the facts sit in the sequent over skolem constants; `f(x)` and `length(l)` become unknown reals, and that `length(l)` is nonnegative comes from its type. The strategy says which formulas it uses, which terms become unknowns, and which sentence it decides:

```
unknown_terms :
|-----
{1}  FORALL (f: [real -> real], l: list[real], x: real):
      f(x) > x ^ 2 AND length(l) = 3 IMPLIES f(x) + length(l) > 2
Rule? (skeep)
{-1} f(x) > x ^ 2
{-2} length(l) = 3
|-----
{1}  f(x) + length(l) > 2
Rule? (cad *)
cad*: using (-1 -2 1), skipping nothing; unknown reals: cadv_1 = f(x), cadv_2 = x, cadv_3 = length(l)
cad*: deciding FORALL (cadv_1: real): FORALL (cadv_2: real): FORALL (cadv_3: real): (((cadv_3 >= 0))
  ↳ IMPLIES (((cadv_1 > (cadv_2 ^ 2)) AND (cadv_3 = (3))) IMPLIES (((cadv_1 + cadv_3) > (2)))))
Deciding * by cylindrical decision, witness first,
Q.E.D.
```

Quantifier elimination inside a proof: `(cad-qe -1)` prints the quantifier-free answer and replaces the hypothesis by it, with a proof that the two are equivalent; `(cad *)` finishes:

```

qe_then_cad :
|-----
{1}  FORALL (a, b: real):
      (EXISTS (t: real): t ^ 2 + a * t + b = 0) AND b > 0 IMPLIES
      a ^ 2 >= 4 * b
Rule? (skeep)
{-1}  EXISTS (t: real): t ^ 2 + a * t + b = 0
{-2}  b > 0
|-----
{1}  a ^ 2 >= 4 * b
Rule? (cad-qe -1)
cad-qe: ((b < 0) OR ((b = 0) OR ((b > 0) AND ((4*b + (-1)*a^2 = 0) OR (4*b + (-1)*a^2 < 0)))))
Eliminating the quantifiers of -1,
{-1}  ((b < 0) OR
      ((b = 0) OR
      ((b > 0) AND
      ((4 * b + (-1) * a ^ 2 = 0) OR (4 * b + (-1) * a ^ 2 < 0)))))
{-2}  b > 0
|-----
{1}  a ^ 2 >= 4 * b
Rule? (cad *)
cad*: using (-1 -2 1), skipping nothing; unknown reals: cadv_1 = b, cadv_2 = a
cad*: deciding FORALL (cadv_1: real): FORALL (cadv_2: real): (((cadv_1 < (0)) OR ((cadv_1 = (0)) OR
  ↳ ((cadv_1 > (0)) AND (((4) * cadv_1 + ((-1) * (cadv_2 ^ 2))) = (0)) OR (((4) * cadv_1 + ((-1) *
  ↳ (cadv_2 ^ 2))) < (0))))) AND (cadv_1 > (0))) IMPLIES (((cadv_2 ^ 2) >= ((4) * cadv_1)))
Deciding * by cylindrical decision, witness first,
Q.E.D.

```

How a proof by (cad) works

The strategy follows the reflection template of Narkawicz, Muñoz and Dutle’s univariate Sturm and Tarski strategies in NASALib [4, 5]. (cad) reads the goal and computes its encoding as data (os, F, phi): the quantifier prefix, the polynomials’ coefficient lists and the Boolean skeleton (see “The main theorems”). It then runs the decision in PVS’s ground evaluator (decide8, the Collins run, in a theory that imports cad_decide8, as pvs_cad does) and instantiates its correctness theorem (decide8_correct) with that data. Unfolding fsem, the semantics, turns the result back into the original formula: each quantifier is matched, and one reflection lemma per polynomial (peval_pnorm) shows that the coefficient list evaluates to the polynomial as written. A FALSE sentence is reported and left in place, and a false hypothesis closes the goal. Before deciding a prenex sentence, (cad) looks for a witness: a model search proves a goal whose quantifiers are all existential (or refutes a hypothesis whose quantifiers are all universal) by instantiating a point, and with three or more quantifiers, when the decision is slow, one variable (existential in a goal, universal in a hypothesis being refuted) is fixed to a small constant and the smaller sentence decided. Grouped binders such as FORALL (x, y: real) are decided through the nested form, and a goal NOT A as the hypothesis A. In a theory that imports pvs_cad, a formula of any other shape (quantifiers inside connectives; binders over posreal, nreal, negreal, npreal or nzreal) is encoded as a term g of a datatype that mirrors PVS’s connectives; (cad) evaluates decide_g(g), instantiates decide_g_ok, unfolds the semantics gsem, which then has the formula’s own shape, and closes the goal connective by connective. (cad-qe) does the same for a formula with free real terms (skolem constants, say): it evaluates qe8, or qe_g for other shapes, and replaces the formula by the quantifier-free answer, proved equivalent from qe8_correct or qe_g-ok. (cad *) collects the polynomial hypotheses and goals of a whole sequent, replaces non-polynomial real terms such as sqrt(x) or f(x) by universally quantified reals, and decides the implication, which enters the proof by case, so both branches are checked by PVS. When that does not prove the goal, (cad) adds what pins such terms down and decides again (cad-num). sqrt, abs, max, min and quotients get the formulas that fix them exactly (sqrt(t) >= 0 and sqrt(t) * sqrt(t) = t, the definitions as two cases, (t / u) * u = t). With IMPORTING pvs_cad_num (pvs_cad, trans_bounds and

NASALib’s `interval_arith@strategies`, constants that NASALib’s interval arithmetic evaluates (π , e , $\sin 1$, $\ln 2$, ...) get enclosures between two rationals, proved by (`numerical`), with the precision raised until the goal is proved, or until it is false throughout the enclosures (by default up to 24 decimals); `sin`, `cos`, `exp`, `ln` and `atan` of terms with constants of the sequent get proved polynomial bounds ($t - t^3/6 \leq \sin t \leq t$ for $t \geq 0$, $1 + t \leq e^t$, ...), then enclosures over pieces of the ranges that the hypotheses give the constants, each piece decided in its own branch and halved where the decision fails (by default at most 5 times; `cad-num`’s precisions and `splits` change both limits; a goal’s leading universal quantifiers are skolemized first). All of these enter as proved hypotheses: the exact facts by lemmas, the enclosures by NASALib’s (`numerical`), whose interval evaluations run in the ground evaluator (next paragraph). Under a binder, a term such as `sqrt(x^2 + 1)` is read into an equivalent formula without it (`FORALL (s: nreal): s * s = x^2 + 1 IMPLIES ...; q * u = t` for a quotient t / u ; the cases of `abs`, `max` and `min`), which is decided, and the original is proved from it by the same mirror walk as for formulas of any shape.

The trusted base is the PVS kernel plus PVSio’s ground evaluator, for every evaluation the strategies ask for: the procedure (`decide8`, `decide_g`, `qe8` or `qe_g`), the normal forms the reflection lemmas compare (`pnorm`), the witness points of the model search, and, with `pvs_cad_num`, the interval evaluations of NASALib’s (`numerical`); the same base as NASALib’s Sturm and Tarski strategies. It also includes NASALib’s saved proofs of the theorems the library imports, which were not replayed here. Beyond that, one trusts the specification: that the semantics (`fsem`, `gsem`, `qfsem`) and the theorem statements (last section) say what is intended. The strategy code (3,715 lines of Lisp) is not trusted: it only chooses the proof steps, which the kernel checks.

What it decides in practice

CAD is doubly exponential in the worst case, so completeness means “always answers, given time”. Whole proofs of lemmas of the example theories, each timed by PVS in one `proveit` session on one M-series Mac: in this library (6 October 2026) and, for comparison, in the full development (`v1.0-full`, 5 October 2026), whose (`cad`) uses the same Collins run:

Sentence	Shape	v1.0-full	this library
Wilkinson’s $(x - 1) \cdots (x - 20)$, expanded, is positive for $x < 1$ (coefficients up to 1.4×10^{19})	$\forall$, degree 20	6.0 s	5.4 s
Chebyshev T_{30} stays in $[-1, 1]$ on $[-1, 1]$	$\forall$, degree 30	8.3 s	7.5 s
Motzkin’s polynomial is nonnegative (not a sum of squares [14])	$\forall\forall$, degree 6	1.7 s	1.1 s
Every cubic $x^3 + px + q$ has a real root	$\forall\exists$	1.7 s	1.0 s
Quadratic formula: $a \neq 0 \wedge b^2 \geq 4ac \Rightarrow \exists x : ax^2 + bx + c = 0$	$\forall\forall\exists$	8.1 s	2.4 s
x^2 is continuous at 0 (ϵ - δ , binders over <code>posreal</code>)	$\forall\exists$	2.2 s	1.8 s
AM-GM for three numbers, $x^3 + y^3 + z^3 \geq 3xyz$	$\forall\forall\forall$, degree 3	2.8 s	2.4 s
Schur’s inequality ($t = 1$)	$\forall\forall\forall$, degree 3	3.7 s	2.9 s
Cauchy-Schwarz in the plane	$\forall\forall\forall\forall$, degree 4	1.9 s	1.4 s
$f(x) > x^2 \wedge \text{length}(l) = 3 \Rightarrow f(x) + \text{length}(l) > 2$, by (<code>cad *</code>) after (<code>skeep</code>) (the session above)	unknown terms	3.6 s	2.0 s
No real root of $z^2 + bz + c$ implies $b^2 < 4c$, by (<code>cad *</code>) on a sequent	quantified hypothesis	2.6 s	1.5 s
(<code>cad-qe</code>) on $\exists t : t^2 + at + b = 0$, then (<code>cad *</code>) (the session above)	elimination, then decision	7.6 s	4.0 s

Mignotte’s polynomial (two roots about 10^{-7} apart) and AM-GM for four numbers (degree 4) do not finish in two minutes (4 October 2026). The 466 proofs of the 14 timed theories (the examples and the Bath problems) that both versions have took 1,032 s in that session, against 1,660 s in the full development (−38%), and none was slower by more than 0.02 s (`docs/collins_cad.pdf` has the times per theory).

How it was built

The library was built in stages from 9 September to 5 October 2026 (315 commits, the full development, tag `v1.0-full`), and then reduced for NASALib to what its procedures, their theorems and the examples use (5–6 October 2026, 20 commits).

1. **Multivariate polynomials.** A normal form for multivariate polynomials over the rationals, proved unique, and reflection lemmas connecting it to PVS real expressions, which the strategies use to read a formula's polynomials.
2. **Real algebraic numbers.** Root isolation, ordering and signs at algebraic numbers, on top of NASALib's Sturm and Tarski libraries.
3. **Resultants and subresultants:** determinants, the resultant, and the principal subresultant coefficients with their specialization property, from which Collins's N-reducta projection is built (item 5).
4. **Earlier engines** (in the full development only). A first cylindrical decision, `decide`, was proved correct for every sentence; it was complete but exponential in the number of polynomials (a circle and a line did not finish in 15 minutes). The read-closure engines that followed (`decide5`, `decide7`) lift cell by cell, reading signs through Tarski queries at exact, possibly algebraic, sample points, with run-time certificates; they were proved sound, complete (every large enough value of one search-effort parameter makes every check pass) and, whenever their certificate holds, to compute a CAD, and they had a quantifier elimination of their own. Their walk over exact sample points is the one the Collins run uses (`walk_def`, `walk_ok`); the rest was removed when the Collins run, whose delineability needs no certificate, replaced them.
5. **Collins's projection.** Collins's N-reducta projection (`projn`, executable) is proved delineating for every input, by the subresultant theorem and the continuity of complex roots in the coefficients (`collins_stack`, `projn_ok`). The cells built from its tower form a CAD adapted to the family in every number of variables, with no run-time certificate (`col_cells`), which `col_found` returns as data (`col_found_cad`). `decide8` decides on that tower (with the line cut at the roots of a smaller bottom step, item 8), and `qe8` eliminates quantifiers on it or on one whose free levels are closed under the derivative (`decide8_decides`, `qe8_complete`).
6. **Tarski–Seidenberg.** `qelim` eliminates the quantifiers of every first-order formula from the inside out, each existential quantifier by `qe8`, or by `decide8` when no variable is free (`qelim_qf`, `qelim_ok`). So a set is first-order definable over $\mathbb{Q}$ exactly when it is quantifier-free definable (`fod_qfd`), and every cell of the CAD is (`col_cells_sa`).
7. **Formulas as written.** A datatype `Gm` mirrors PVS's connectives; `decide_g` decides its sentences and `qe_g` eliminates its quantifiers, both through `qelim` (`decide_g_ok`, `qe_g_ok`). With `IMPORTING pvs_cad`, the one import (`cad`) and (`cad-qe`) need, (`cad`), (`cad *`) and (`cad-qe`) take formulas as written: grouped binders, quantifiers inside NOT, AND, OR, IMPLIES and IFF, and binders over `posreal`, `nnreal`, `negreal`, `npreal` and `nzreal` (a binder over any other type, such as `nat`, is refused).
8. **A smaller projection, faster determinants.** The subresultant determinants compute each minor once, looked up in a trie (`det_fast`), and then by fraction-free elimination with every exact division checked (`bdet_det`). Each family of the tower is replaced by a basis of it, meant to be squarefree, that a run-time check confirms factors every member (`fsplit_cinv`, `fsplit_nz`). `decide8` cuts the line of the outermost variable at the roots of a smaller bottom step instead of Collins's (`projb`, delineating over a connected set where its members are nonzero, `projb_delin`): for the Bath problem `bath_04`, 35 polynomials instead of 569 in a prototype count. Its cells still form a CAD adapted to its family (`decb_cad_run`), and `decide8` stays sound and complete (`decb_correct`, `decb_ev`).
9. **Constants and other terms.** (`cad`) pins down terms that are not polynomial before it decides: constants such as π and e by enclosures from NASALib's interval arithmetic; `sqrt`, `abs`, `max`, `min` and quotients by the facts that fix them exactly, and under binders by an equivalent formula without them; `sin`, `cos`, `exp`, `ln` and

atan of terms with constants by proved polynomial bounds (`trans_bounds`) and by enclosures over pieces of the constants' ranges (`cad-num`).

The library in numbers

PVS theories of the library	210	Formulas proved, whole-library replay	3,175 of 3,175
Lines of PVS (213 files, with the datatypes)	15,254	lemmas and theorems	2,031
Lines of strategy code (Lisp)	3,715	TCCs (type-correctness obligations)	1,144
Example theories (<code>cad/cad_examples</code>)	14	Formulas proved in the examples	457 of 457
Commits, from 9 September 2026	335	Axioms added, beyond PVS's datatype axioms	none

It builds on NASALib's Sturm and Tarski (univariate root counting) and on reals, structures, analysis, complex, matrices, `interval_arith`, `trig` and `lnexp`.

Where this sits

Tarski showed that the first-order theory of the real numbers is decidable [1], and Collins's cylindrical algebraic decomposition [2] made the decision practical; this library builds on the PVS [3] libraries of NASALib [4], in particular the univariate Sturm and Tarski decision procedures of Narkawicz, Muñoz and Dutle [5]. Earlier formally verified or proof-producing quantifier elimination for the reals includes, by methods other than CAD, Cohen and Mahboubi's verified procedure in Coq [6], McLaughlin and Harrison's proof-producing Cohen–Hörmander procedure in HOL Light [7], and a verified, complete multivariate procedure in Isabelle/HOL [8], exported to SML code which, without virtual substitution, in its authors' words “appears to hang on all but the simplest univariate examples”. Mahboubi implemented CAD in Coq without a completed correctness proof [9]. A formal correctness proof of Collins's CAD, including the delineability of Collins's projection, exists in Rocq [10], over an abstract real closed field; it also yields a verified Boolean decision procedure for systems of strict polynomial inequalities, but none of it runs as a tactic. Lean's Mathlib [11] has, as far as we could find, no CAD formalization and no CAD-based decision tactic: an issue of the Hex project (Lean), opened in September 2026 to commission a roadmap, lists Sturm theory, Thom's lemma, delineability and CAD among what Mathlib lacks [12]. Lean's nonlinear tactics are not complete: `nlinarith` multiplies hypotheses and adds squares before calling `linarith`, `grobner` proves polynomial equalities by Gröbner bases, and `polyrith`, which checked a certificate found by Sage, is no longer supported; the Hex project's `rcf` tactic decides one-variable sentences with kernel-checked certificates. To our knowledge (a survey of published work and public repositories, September–October 2026), `pvs_cad` is the first CAD-based decision procedure for the full first-order theory of the reals, in any number of variables, that is executable, formally verified sound and complete, and usable as a tactic in an interactive prover; its answers become PVS proofs by computational reflection, trusting the PVS kernel and its ground evaluator, as NASALib's Sturm and Tarski strategies do. The theorems about its CAD state the conditions of Basu, Pollack and Roy's Defs. 5.1 and 5.5 [13] (a cylindrical decomposition adapted to the polynomials), in two respects only indirectly: the partition is stated for the cells of $\mathbb{R}^n$, and at lower levels it follows from the stack construction (`tcell_cover`, `tcell_disj`) without being restated; finiteness comes through the records of `col_found`, at least one for every nonempty cell. What is proved, and what is not:

- “Semi-algebraic” holds in two senses. The CAD theorems state that every cell, at every level, and every root function's graph over a cell is defined by a first-order formula whose atoms are sign conditions on polynomials with rational coefficients. `fod_qfd` (`qelim_ok.pvs`) proves that a set is defined by such a formula exactly when a Boolean combination of such sign conditions defines it without quantifiers (Tarski–Seidenberg over $\mathbb{Q}$), and `cad_sa.pvs` states this for every cell of the CAD, at every level, and for its root graphs (`col_cells_sa`). Sets so defined over $\mathbb{Q}$ are semi-algebraic in Basu, Pollack and Roy's sense, whose definition also allows real coefficients and so admits more sets ($\{\pi\}$, for one). No tool prints the quantifier-free definitions of the cells; the theorems say that they exist.

- Delineability is proved for Collins’s N-reducta projection, for every input (`collins_stack`, `col_stack.pvs`); `projn_ok` ties it to the executable operator `projn`, whose tower, each family replaced by a checked basis of it, `decide8` and `qe8` use; `decide8` cuts the line of the outermost variable at the roots of a smaller bottom step, `projb`, which is delineating over a connected set where its members are nonzero (`projb_delin`). McCallum’s and Brown–McCallum’s smaller projections are not proved delineating, and Lazard’s is not done. Delineability is in the sign-invariant sense (`delin_cl?`): root multiplicities are not tracked.
- The Collins CAD theorems cover every number of variables, one included (`col_cells`; in one variable the cells are the sectors of the real roots of F , `col_cad_line`, the sectors over which `decide8` decides one-variable sentences).
- The CAD theorems cover the Collins run and `decide8`’s run with the smaller bottom step (`decb_cad_run`). (`cad`) also has a shortcut they do not cover, the witness search; its answers are proved correct too. (`cad :cad-only? t`) does not use it, in its side decisions either: it decides by `decide8`, and by `decide_g` for formulas of other shapes.

The main theorems

A prenex sentence is encoded as data: `os` lists the quantifiers outermost first (TRUE for $\forall$, FALSE for $\exists$), `F` lists the polynomials (exact rational coefficients), and `phi` is the Boolean skeleton, whose atoms say “polynomial i has sign s ”. Its meaning is defined by recursion on the prefix, with the real quantifiers of PVS itself:

The semantics (`cad_decide.pvs`)

```
fsem(os, F, phi, ys): RECURSIVE bool =
  CASES os OF
    null: bfeval(phi, F, ys),
    cons(q, r): IF q THEN FORALL x: fsem(r, F, phi, cons(x, ys))
               ELSE EXISTS x: fsem(r, F, phi, cons(x, ys)) ENDIF
  ENDCASES
```

The decision procedures return a record: `val`, the verdict, and `ok`, a flag that is TRUE when the run-time checks passed. `decide8` is the Collins run, over a checked basis of the input family and with the smaller bottom step:

Soundness and completeness (`cad_decide8.pvs`)

```
decide8_correct: THEOREM decide8(reverse(os), F, phi)`ok IMPLIES
                  (decide8(reverse(os), F, phi)`val IFF fsem(os, F, phi, null))
decide8_decides: THEOREM decide8(reverse(os), F, phi)`val IFF fsem(os, F, phi, null)
```

There are no hypotheses: `os`, `F` and `phi` are arbitrary, so the theorems cover every closed prenex sentence with any number of quantifiers. `decide8` is a total function, and PVS generated and proved its termination obligations, so it always terminates and returns the truth of the sentence.

Quantifiers anywhere need another formula type, `rcf_fol`’s F_m (`ftrue`, sign atoms `fsg(p, s)`, `fnot`, `fand`, `fex`; variables are positions in a point). Its semantics `fsem(phi)(pt)` is a different function from `fsem(os, F, phi, ys)` above; `qfsem` is the same semantics under another name, and `qf?(phi)` says that `phi` has no `fex`. `gform_def`’s G_m mirrors PVS’s connectives (TRUE, FALSE, the six comparisons of two polynomials, NOT, AND, OR, IMPLIES, IFF, FORALL, EXISTS) and its meaning `gsem(g)(pt)` is written with them; `decide_g(g)` and `qe_g(g, m)` apply `qelim` to its translation into F_m . `qelim` works from the inside out: each `fex` by `qe8`, or by `decide8` when no variable is free.

In the theorems below on `g` and `qelim` the formula is arbitrary and `m`, where it occurs, is any natural number (`m = 0`: a sentence). In `qe8_complete`, `m` is a positive natural number and `cons?(os)` asks for at least one

quantifier: every prenex formula with at least one quantifier and $m \geq 1$ free variables. The atoms' polynomials have rational coefficients and the formulas have no parameters, so `fod_qfd` is the Tarski–Seidenberg theorem for formulas over $\mathbb{Q}$.

Any shape, quantifier elimination, Tarski–Seidenberg (`gform_ok`, `qe8`, `qelim_ok`, `rcf_fol`)

```

decide_g_ok: THEOREM decide_g(g) IFF gsem(g)(null)
qe_g_qf: LEMMA qf?(qe_g(g, m))
qe_g_ok: THEOREM length(pt) = m IMPLIES (fsem(qe_g(g, m))(pt) IFF gsem(g)(pt))
qe8_complete: THEOREM cons?(os) AND length(pt) = m IMPLIES
  qf?(qe8(m, os, F, phi)`out) AND (qfsem(qe8(m, os, F, phi)`out)(pt) IFF fsem(os, F, phi, pt))
qelim_qf: LEMMA FORALL phi: FORALL m: qf?(qelim(phi, m))
qelim_ok: THEOREM FORALL phi: FORALL m, pt: length(pt) = m
  IMPLIES (fsem(qelim(phi, m))(pt) IFF fsem(phi)(pt))
fod?(n)(S): bool = EXISTS phi: FORALL pt: length(pt) = n IMPLIES (S(pt) IFF fsem(phi)(pt))
qfd?(n)(S): bool = EXISTS phi: qf?(phi) AND FORALL pt: length(pt) = n IMPLIES (S(pt) IFF fsem(phi)(pt))
fod_qfd: THEOREM fod?(n)(S) IFF qfd?(n)(S)

```

The Collins CAD (`col_stack.pvs`, `col_line.pvs`, `col_found_ok.pvs`, `cad_sa.pvs`)

```

collins_stack: THEOREM conn?(C) AND C(p0) AND colh?(G, C) IMPLIES delin_cl?(G, C)
col_cells: THEOREM ccells?(k, F)
col_found_cad: THEOREM cons?(qs) IMPLIES cfound?(qs, F, col_found(qs, F)`freces)
col_cells_sa: THEOREM
  (FORALL s, A: member(s, csects(k, F)) AND length(A) <= k
    IMPLIES qfd?(1 + length(A))(ccell(k, F)(s, A)))
  AND (FORALL s, A, i: member(s, csects(k, F)) AND length(A) < k
    AND (FORALL q: ccell(k, F)(s, A)(q) IMPLIES i < nroot(nth(ctw(F, k), length(A)), q))
    IMPLIES qfd?(2 + length(A))
    (LAMBDA p: cons?(p) AND ccell(k, F)(s, A)(cdr(p))
      AND car(p) = rt(nth(ctw(F, k), length(A)), cdr(p), i)))

```

`colh?(G, C)` says that Collins's projection of G has one sign on C : the leading coefficients of the kept reducta, their principal subresultant coefficients with their derivatives, and those of pairs of members; `projn_ok` (`cad_projn.pvs`) derives it from the sign-invariance of the executable operator `projn`. `ccells?(k, F)` states, for the cells `ccell(k, F)(s, A)` built from the tower `ctw(F, k)` of Collins projections of a family F in $k + 1$ variables, that the cells of $\mathbb{R}^{k+1}$ cover it and are pairwise disjoint, that every cell at every level is connected and first-order definable, that over each sector of the line every level is delineable over every cell below it, that the graph of every root function over a cell is definable, and that F has one sign vector on every cell of $\mathbb{R}^{k+1}$; `col_cells` has no hypothesis: the cells form a CAD with no run-time check (`col_found` and `decide8` still check their walk's certificate while they raise u , which always ends: `decc_ev`, `decb_ev`). `decb_cad_run` (`decb_run.pvs`) states the same conditions for the cells of `decide8`'s run, whose outer line is cut at the roots of the smaller bottom step when every member is nonzero at the sample of every sector that is not a root (`nzb?`), and `decb_cad_input` gives the input family one sign vector on every cell of the run over its checked basis. `cfound?(qs, F, L)` adds the records L : each lies in its cell, with an exact sample point and F 's signs there, and every nonempty cell of $\mathbb{R}^n$, $n = \text{length}(qs)$, has at least one; by `col_found_decides` they decide every sentence over F in the same variables, in the same order. `col_cells_sa` states that every cell, at every level, and every root function's graph over a cell is quantifier-free definable.

References

- [1] A. Tarski. *A Decision Method for Elementary Algebra and Geometry*. RAND Corporation, 1948; 2nd ed., University of California Press, 1951.
- [2] G. E. Collins. Quantifier elimination for real closed fields by cylindrical algebraic decomposition. In *Automata Theory and Formal*

- Languages*, LNCS 33, pages 134–183. Springer, 1975. https://doi.org/10.1007/3-540-07407-4_17
- [3] S. Owre, J. M. Rushby, N. Shankar. PVS: A prototype verification system. In *CADE-11*, LNAI 607, pages 748–752. Springer, 1992. https://doi.org/10.1007/3-540-55602-8_217
 - [4] NASALib, the NASA PVS library collection. <https://github.com/nasa/pvslib>
 - [5] A. Narkawicz, C. Muñoz, A. Dutle. Formally-verified decision procedures for univariate polynomial computation based on Sturm’s and Tarski’s theorems. *Journal of Automated Reasoning* 54(4):285–326, 2015. <https://doi.org/10.1007/s10817-015-9320-x>
 - [6] C. Cohen, A. Mahboubi. Formal proofs in real algebraic geometry: from ordered fields to quantifier elimination. *Logical Methods in Computer Science* 8(1:02), 2012. [https://doi.org/10.2168/LMCS-8\(1:2\)2012](https://doi.org/10.2168/LMCS-8(1:2)2012)
 - [7] S. McLaughlin, J. Harrison. A proof-producing decision procedure for real arithmetic. In *CADE-20*, LNCS 3632, pages 295–314. Springer, 2005. https://doi.org/10.1007/11532231_22
 - [8] K. Kosaian, Y. K. Tan, A. Platzer. A first complete algorithm for real quantifier elimination in Isabelle/HOL. In *CPP 2023*, pages 211–224. ACM, 2023. <https://doi.org/10.1145/3573105.3575672>
 - [9] A. Mahboubi. Implementing the cylindrical algebraic decomposition within the Coq system. *Mathematical Structures in Computer Science* 17(1):99–127, 2007. <https://doi.org/10.1017/S096012950600586X>
 - [10] Q. Vermande. Cylindrical algebraic decomposition in Coq/Rocq. In *CPP 2026*, pages 45–58. ACM, 2026. <https://doi.org/10.1145/3779031.3779100>
 - [11] The mathlib Community. The Lean mathematical library. In *CPP 2020*, pages 367–381. ACM, 2020. <https://doi.org/10.1145/3372885.3373824>
 - [12] K. Morrison. A roadmap for the real algebraic geometry behind cylindrical algebraic decomposition. Hex project (Lean), GitHub issue 10300, 20 September 2026. <https://github.com/kim-em/hex-dev/issues/10300>
 - [13] S. Basu, R. Pollack, M.-F. Roy. *Algorithms in Real Algebraic Geometry*, 2nd ed. Springer, 2006.
 - [14] T. S. Motzkin. The arithmetic-geometric inequality. In O. Shisha (ed.), *Inequalities*, pages 205–224. Academic Press, 1967.